```python
import numpy as np



nodes = ['S', 'A', 'B', 'C', 'D', 'BS']

energy = np.array([
    [0, 4, 6, 0, 0, 0],
    [4, 0, 0, 5, 8, 0],
    [6, 0, 0, 0, 4, 0],
    [0, 5, 0, 0, 0, 6],
    [0, 8, 4, 0, 0, 5],
    [0, 0, 0, 6, 5, 0]
])

signal = np.array([
    [0, 90, 80, 0, 0, 0],
    [90, 0, 0, 85, 75, 0],
    [80, 0, 0, 0, 88, 0],
    [0, 85, 0, 0, 0, 92],
    [0, 75, 88, 0, 0, 87],
    [0, 0, 0, 92, 87, 0]
])

battery = np.array([100, 90, 85, 80, 75, 100])

source = 0
destination = 5


def fitness(position):

    route = [source]
    current = source
```

```
destination = 5


def fitness(position):

    route = [source]
    current = source
    visited = {source}

    while current != destination:

        neighbors = [
            j for j in range(len(nodes))
            if energy[current][j] > 0 and j not in visited
        ]

        if not neighbors:
            return 1000

        next_node = neighbors[
            np.argmin(position[neighbors])
        ]

        route.append(next_node)
        visited.add(next_node)
        current = next_node

    total_energy = 0
    total_signal = 0
    battery_penalty = 0

    for i in range(len(route) - 1):

        u = route[i]
        v = route[i + 1]

        total_energy += energy[u][v]
        total_signal += signal[u][v]
```

```
            u = route[i]
            v = route[i + 1]

            total_energy += energy[u][v]
            total_signal += signal[u][v]
            battery_penalty += (100 - battery[u])

        average_signal = total_signal / (len(route) - 1)

        cost = (
            total_energy
            + (100 - average_signal) * 0.2
            + battery_penalty * 0.02
        )

        return cost



n_wolves = 10
iterations = 10
n_nodes = len(nodes)

np.random.seed(42)

positions = np.random.rand(n_wolves, n_nodes)

alpha_score = float('inf')
beta_score = float('inf')
delta_score = float('inf')

alpha_pos = None
beta_pos = None
delta_pos = None


for t in range(iterations):
```

```
for t in range(iterations):

    for i in range(n_wolves):

        score = fitness(positions[i])

        if score < alpha_score:

            delta_score = beta_score
            delta_pos = beta_pos

            beta_score = alpha_score
            beta_pos = alpha_pos

            alpha_score = score
            alpha_pos = positions[i].copy()

        elif score < beta_score:

            delta_score = beta_score
            delta_pos = delta_pos

            beta_score = score
            beta_pos = positions[i].copy()

        elif score < delta_score:

            delta_score = score
            delta_pos = positions[i].copy()

    a = 2 - (2 * t / iterations)

    for i in range(n_wolves):

        r1 = np.random.rand(n_nodes)
        r2 = np.random.rand(n_nodes)

        A1 = 2 * a * r1 - a
```

```
    for i in range(n_wolves):

        r1 = np.random.rand(n_nodes)
        r2 = np.random.rand(n_nodes)

        A1 = 2 * a * r1 - a
        C1 = 2 * r2

        D_alpha = abs(C1 * alpha_pos - positions[i])
        X1 = alpha_pos - A1 * D_alpha

        r1 = np.random.rand(n_nodes)
        r2 = np.random.rand(n_nodes)

        A2 = 2 * a * r1 - a
        C2 = 2 * r2

        D_beta = abs(C2 * beta_pos - positions[i])
        X2 = beta_pos - A2 * D_beta

        r1 = np.random.rand(n_nodes)
        r2 = np.random.rand(n_nodes)

        A3 = 2 * a * r1 - a
        C3 = 2 * r2

        D_delta = abs(C3 * delta_pos - positions[i])
        X3 = delta_pos - A3 * D_delta

        positions[i] = np.clip(
            (X1 + X2 + X3) / 3, 0, 1
        )

    print(
        "Iteration:", t + 1,
        "| Best Cost:", round(alpha_score, 2)
    )
```

```
            )

        print(
            "Iteration:", t + 1,
            "| Best Cost:", round(alpha_score, 2)
        )


route = [source]
current = source
visited = {source}

while current != destination:

    neighbors = [
        j for j in range(n_nodes)
        if energy[current][j] > 0 and j not in visited
    ]

    next_node = neighbors[
        np.argmin(alpha_pos[neighbors])
    ]

    route.append(next_node)
    visited.add(next_node)
    current = next_node

total_energy = 0
total_signal = 0

print("\n----- FINAL RESULTS -----")

print("Best Communication Route:")

print(" -> ".join(nodes[i] for i in route))

for i in range(len(route) - 1):
```

```
        ]

        route.append(next_node)
        visited.add(next_node)
        current = next_node

    total_energy = 0
    total_signal = 0

    print("\n----- FINAL RESULTS -----")

    print("Best Communication Route:")

    print(" -> ".join(nodes[i] for i in route))

    for i in range(len(route) - 1):

        u = route[i]
        v = route[i + 1]

        total_energy += energy[u][v]
        total_signal += signal[u][v]

    print("Total Energy Consumption:", total_energy)

    print(
        "Average Signal Quality:",
        round(total_signal / (len(route) - 1), 2), "%"
    )

    print("\nRemaining Battery Power:")

    for node in route[1:-1]:
        print(nodes[node], ":", battery[node], "%")

    print("\nOptimization Completed Successfully!")
```

```
print(
    "Average Signal Quality:",
    round(total_signal / (len(route) - 1), 2), "%"
)

print("\nRemaining Battery Power:")

for node in route[1:-1]:
    print(nodes[node], ":", battery[node], "%")

print("\nOptimization Completed Successfully!")
```

```
Iteration: 1 | Best Cost: 17.8
Iteration: 2 | Best Cost: 17.8
Iteration: 3 | Best Cost: 17.8
Iteration: 4 | Best Cost: 17.8
Iteration: 5 | Best Cost: 17.8
Iteration: 6 | Best Cost: 17.8
Iteration: 7 | Best Cost: 17.8
Iteration: 8 | Best Cost: 17.8
Iteration: 9 | Best Cost: 17.8
Iteration: 10 | Best Cost: 17.8

----- FINAL RESULTS -----
Best Communication Route:
S -> A -> C -> BS
Total Energy Consumption: 15
Average Signal Quality: 89.0 %

Remaining Battery Power:
A : 90 %
C : 80 %

Optimization Completed Successfully!
```